

Motivating the Study of Algorithms, RAM Model and Sorting in Quadratic Time

Dr. Bugra Caskurlu¹

¹ School of Computing
New Uzbekistan University

August 25, 2025

What are Algorithms?

Definition

- An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a finite amount of time. It is thus a sequence of computational steps that transform the input into the output.

What are Algorithms?

Definition

- An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a finite amount of time. It is thus a sequence of computational steps that transform the input into the output.
- An **algorithm** is a tool for solving a well-specified **computational problem**. The statement of the problem specifies the desired input/output relationship for problem instances, typically of arbitrarily large size. The algorithm describes a specific computational procedure for achieving that input/output relationship for all problem instances.

What are Algorithms?

Definition

- An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a finite amount of time. It is thus a sequence of computational steps that transform the input into the output.
- An **algorithm** is a tool for solving a well-specified **computational problem**. The statement of the problem specifies the desired input/output relationship for problem instances, typically of arbitrarily large size. The algorithm describes a specific computational procedure for achieving that input/output relationship for all problem instances.
- **Problem Example:** Suppose that you need to sort a sequence of numbers into monotonically increasing order. We formally define the **sorting problem** as follows:
 - ❶ **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

What are Algorithms?

Definition

- An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output** in a finite amount of time. It is thus a sequence of computational steps that transform the input into the output.
- An **algorithm** is a tool for solving a well-specified **computational problem**. The statement of the problem specifies the desired input/output relationship for problem instances, typically of arbitrarily large size. The algorithm describes a specific computational procedure for achieving that input/output relationship for all problem instances.
- **Problem Example:** Suppose that you need to sort a sequence of numbers into monotonically increasing order. We formally define the **sorting problem** as follows:
 - 1 **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
 - 2 **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Such an input sequence is called an **instance** of the sorting problem. An instance of a problem consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Such an input sequence is called an **instance** of the sorting problem. An instance of a problem consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.
- An algorithm for a computational problem is **correct** if, for every problem instance provided as input, it **halts**, i.e., finishes its computing in finite time, and outputs the correct solution to the problem instance.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Such an input sequence is called an **instance** of the sorting problem. An instance of a problem consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.
- An algorithm for a computational problem is **correct** if, for every problem instance provided as input, it **halts**, i.e., finishes its computing in finite time, and outputs the correct solution to the problem instance.
- A correct algorithm **solves** the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Such an input sequence is called an **instance** of the sorting problem. An instance of a problem consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.
- An algorithm for a computational problem is **correct** if, for every problem instance provided as input, it **halts**, i.e., finishes its computing in finite time, and outputs the correct solution to the problem instance.
- A correct algorithm **solves** the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.
- Incorrect algorithms can sometimes be useful, if you can control their error rate. We will focus on correct algorithms in this class.

Elaborating the Definition of Algorithms

Some Additional Terms

- Given the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$, a correct sorting algorithm returns as output the sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Such an input sequence is called an **instance** of the sorting problem. An instance of a problem consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.
- An algorithm for a computational problem is **correct** if, for every problem instance provided as input, it **halts**, i.e., finishes its computing in finite time, and outputs the correct solution to the problem instance.
- A correct algorithm **solves** the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.
- Incorrect algorithms can sometimes be useful, if you can control their error rate. We will focus on correct algorithms in this class.
- Sorting is a fundamental operation in computer science. As a result, we have a large number of good sorting algorithms. Which algorithm is best for a given application depends on the number of items to be sorted, the extent to which the items are already somewhat sorted, possible restrictions on the item values, the architecture of the computer, and the kind of storage devices to be used.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.
- You would probably want your implementation to be within the bounds of good software engineering practice (i.e., well-designed and documented), but you would use the algorithm which is easiest to implement.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.
- You would probably want your implementation to be within the bounds of good software engineering practice (i.e., well-designed and documented), but you would use the algorithm which is easiest to implement.
- Computers may be fast, but they are not infinitely fast.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.
- You would probably want your implementation to be within the bounds of good software engineering practice (i.e., well-designed and documented), but you would use the algorithm which is easiest to implement.
- Computers may be fast, but they are not infinitely fast. Memory may be cheap, but it is not free.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.
- You would probably want your implementation to be within the bounds of good software engineering practice (i.e., well-designed and documented), but you would use the algorithm which is easiest to implement.
- Computers may be fast, but they are not infinitely fast. Memory may be cheap, but it is not free.
- Computing time is therefore a **bounded** resource, and so is space in memory.

Why we need Algorithms

Motivation

- Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms?
- Yes, if for no other reason than that you would still like to demonstrate that your solution method *terminates* and does so with the *correct* answer.
- However, if computers were infinitely fast, any correct method for solving a problem would do the job.
- You would probably want your implementation to be within the bounds of good software engineering practice (i.e., well-designed and documented), but you would use the algorithm which is easiest to implement.
- Computers may be fast, but they are not infinitely fast. Memory may be cheap, but it is not free.
- Computing time is therefore a **bounded** resource, and so is space in memory. *These resources should be used wisely, and algorithms that are efficient in terms of time or space will help you do so.*

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency.

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .
- **Merge Sort:** Takes roughly $c_2 n \lg n$ time to sort n items, where c_2 is a constant that does not depend on n .

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .
- **Merge Sort:** Takes roughly $c_2 n \lg n$ time to sort n items, where c_2 is a constant that does not depend on n .
- **Fast Computer (A):** 1 billion instructions per second, great implementation of insertion sort, great compiler, so $c_1 = 2$.

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .
- **Merge Sort:** Takes roughly $c_2 n \lg n$ time to sort n items, where c_2 is a constant that does not depend on n .
- **Fast Computer (A):** 1 billion instructions per second, great implementation of insertion sort, great compiler, so $c_1 = 2$.
- **Slow Computer (B):** 10 million instructions per second, poor implementation of merge sort, poor compiler, so $c_2 = 50$.

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .
- **Merge Sort:** Takes roughly $c_2 n \lg n$ time to sort n items, where c_2 is a constant that does not depend on n .
- **Fast Computer (A):** 1 billion instructions per second, great implementation of insertion sort, great compiler, so $c_1 = 2$.
- **Slow Computer (B):** 10 million instructions per second, poor implementation of merge sort, poor compiler, so $c_2 = 50$.
- Sorting an array of size 10^6 by A takes $\frac{2 \cdot (10^6)^2 \text{instructions}}{10^9 \text{instructions/second}} = 2000$ seconds.

Efficiency

Efficiency by Example

- Two different algorithms devised to solve the same problem may differ dramatically in their efficiency. These differences can be much more significant than differences due to hardware and software.
- **Insertion Sort:** Takes roughly $c_1 n^2$ time to sort n items, where c_1 is a constant that does not depend on n .
- **Merge Sort:** Takes roughly $c_2 n \lg n$ time to sort n items, where c_2 is a constant that does not depend on n .
- **Fast Computer (A):** 1 billion instructions per second, great implementation of insertion sort, great compiler, so $c_1 = 2$.
- **Slow Computer (B):** 10 million instructions per second, poor implementation of merge sort, poor compiler, so $c_2 = 50$.
- Sorting an array of size 10^6 by A takes $\frac{2 \cdot (10^6)^2 \text{instructions}}{10^9 \text{instructions/second}} = 2000$ seconds.
- Sorting the same array by B takes $\frac{50 \cdot 10^6 \cdot \lg 10^6 \text{instructions}}{10^7 \text{instructions/second}} = 100$ seconds.

Algorithms and Other Technologies

Relation to Other Technologies

- Our example shows that algorithms, like computer hardware, are a **technology**.

Algorithms and Other Technologies

Relation to Other Technologies

- Our example shows that algorithms, like computer hardware, are a **technology**.
- *Total system performance depends on choosing efficient algorithms more than any other technology especially for large inputs.*

Algorithms and Other Technologies

Relation to Other Technologies

- Our example shows that algorithms, like computer hardware, are a **technology**.
- *Total system performance depends on choosing efficient algorithms more than any other technology especially for large inputs.*
- In this class, we will review and introduce several data structures.

Algorithms and Other Technologies

Relation to Other Technologies

- Our example shows that algorithms, like computer hardware, are a **technology**.
- *Total system performance depends on choosing efficient algorithms more than any other technology especially for large inputs.*
- In this class, we will review and introduce several data structures.
- A **data structure** is a way to store and organize data in order to facilitate access and modifications.

Algorithms and Other Technologies

Relation to Other Technologies

- Our example shows that algorithms, like computer hardware, are a **technology**.
- *Total system performance depends on choosing efficient algorithms more than any other technology especially for large inputs.*
- In this class, we will review and introduce several data structures.
- A **data structure** is a way to store and organize data in order to facilitate access and modifications.
- No data structure works well for all purposes, and so it is important to know the strengths and limitations of several of them and use the one that fits best to our application.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.
- We then remove one card at a time from the table and insert it into the correct position in the left hand.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.
- We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards in the hand, from right to left.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.
- We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards in the hand, from right to left.
- **Property:** At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.
- We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards in the hand, from right to left.
- **Property:** At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.
- *Insertion sort is efficient for sorting a small number of elements.*

Insertion Sort

Sorting Problem Definition

- **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
- **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Explanation of Insertion Sort Algorithm

- **Insertion sort** works the way many people sort a hand of playing cards. We start with an empty left hand and the cards face down on the table.
- We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards in the hand, from right to left.
- **Property:** At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.
- **Insertion sort is efficient for sorting a small number of elements.** The input numbers are **sorted in place**: the numbers are rearranged within the array, with at most a constant number of them stored outside the array at any time.

Insertion Sort

Pseudocode for Insertion Sort

```
INSERTION-SORT (A)
for  $i = 2$  to  $\text{length}[A]$ 
     $\text{key} = A[i];$ 
     $j = i - 1;$ 

    while  $j > 0$  and  $A[j] > \text{key}$ 
         $A[j + 1] = A[j];$ 
         $j = j - 1;$ 

     $A[j + 1] = \text{key};$ 
```

Operation of Insertion Sort

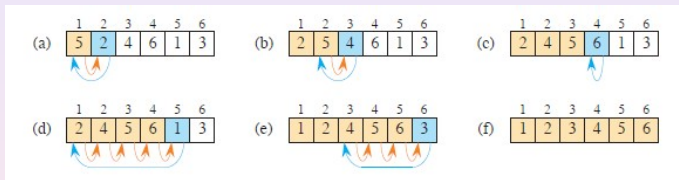


Figure: The operation of $\text{INSERTION-SORT}(A)$, where A initially contains the sequence $\langle 5, 2, 4, 6, 1, 3 \rangle$. Array indices appear above the rectangles, and values stored in the array positions appear within the rectangles. (a) – (e) The iterations of the for loop of lines 1 – 8. In each iteration, the blue rectangle holds the key taken from $A[i]$, which is compared with the values in tan rectangles to its left in the test of line 5. Orange arrows show array values moved one position to the right in line 6, and blue arrows indicate where the key moves to in line 8. (f) The final sorted array.

Correctness

Question

Is INSERTION-SORT a correct sorting algorithm, i.e., does it **terminate** with a **correct output**?

Correctness

Question

Is INSERTION-SORT a correct sorting algorithm, i.e., does it **terminate** with a **correct output**?

Loop Invariants

- In computer science, a **loop** is a programming language statement which allows code to be repeatedly executed.

Correctness

Question

Is INSERTION-SORT a correct sorting algorithm, i.e., does it **terminate** with a **correct output**?

Loop Invariants

- In computer science, a **loop** is a programming language statement which allows code to be repeatedly executed.
- A **loop invariant** is a property that holds before (and after) each repetition.

Correctness

Question

Is INSERTION-SORT a correct sorting algorithm, i.e., does it **terminate** with a **correct output**?

Loop Invariants

- In computer science, a **loop** is a programming language statement which allows code to be repeatedly executed.
- A **loop invariant** is a property that holds before (and after) each repetition.
- A loop invariant is a logical assertion and knowing a loop's invariant is essential for understanding the effect of a loop.

Correctness

Question

Is INSERTION-SORT a correct sorting algorithm, i.e., does it **terminate** with a **correct output**?

Loop Invariants

- In computer science, a **loop** is a programming language statement which allows code to be repeatedly executed.
- A **loop invariant** is a property that holds before (and after) each repetition.
- A loop invariant is a logical assertion and knowing a loop's invariant is essential for understanding the effect of a loop.
- We use invariants to help us understand why an algorithm is correct.

Insertion Sort Loop Invariant

Loop Invariant

We must show three things about a loop invariant:

- **Initialization:** The invariant is true prior to the first iteration of the algorithm.
- **Maintenance:** If the invariant is true before an iteration of a loop, it remains true before the next iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Insertion Sort Loop Invariant

Loop Invariant

We must show three things about a loop invariant:

- **Initialization:** The invariant is true prior to the first iteration of the algorithm.
- **Maintenance:** If the invariant is true before an iteration of a loop, it remains true before the next iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Note the similarity to mathematical induction, where to prove that a property holds, you prove a base case and an inductive step.

Insertion Sort Loop Invariant

Loop Invariant

We must show three things about a loop invariant:

- **Initialization:** The invariant is true prior to the first iteration of the algorithm.
- **Maintenance:** If the invariant is true before an iteration of a loop, it remains true before the next iteration.
- **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Note the similarity to mathematical induction, where to prove that a property holds, you prove a base case and an inductive step.

Insertion Sort

At the start of each iteration of the outer loop (for loop), the subarray $A[1 : i - 1]$ consists of the elements originally in $A[1 : i - 1]$ but in sorted order.

Proving the Invariant

Initialization

We need to show that the loop invariant holds before the first loop iteration, when $i = 2$. The subarray $A[1 : i - 1]$ therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$, and this subarray is trivially sorted.

Proving the Invariant

Initialization

We need to show that the loop invariant holds before the first loop iteration, when $i = 2$. The subarray $A[1 : i - 1]$ therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$, and this subarray is trivially sorted.

Maintenance

The body of the **for** loop works by moving $A[i - 1]$, $A[i - 2]$, and so on by one position to the right until the proper position for $A[i]$ is found, at which point the value of $A[i]$ is inserted.

Proving the Invariant

Initialization

We need to show that the loop invariant holds before the first loop iteration, when $i = 2$. The subarray $A[1 : i - 1]$ therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$, and this subarray is trivially sorted.

Maintenance

The body of the **for** loop works by moving $A[i - 1]$, $A[i - 2]$, and so on by one position to the right until the proper position for $A[i]$ is found, at which point the value of $A[i]$ is inserted.

Termination

The outer **for** loop ends when i exceeds n , i.e., $i = n + 1$. Substituting $n + 1$ for i in the wording of the loop invariant, we have the subarray $A[1 : n]$ consists of the elements originally in $A[1 : n]$, but in sorted order. But the subarray $A[1 : n]$ is the entire array. Hence, the entire array is sorted, which means the algorithm is correct.

Analyzing Algorithms

Definitions

- **Analyzing** an algorithm means *predicting the resources that the algorithm requires*.

Analyzing Algorithms

Definitions

- **Analyzing** an algorithm means *predicting the resources that the algorithm requires*.
- Depending on the application; utilization of memory, computer bandwidth, or computer hardware may be of interest.
- But, in general, **computation time** is the main concern.

Analyzing Algorithms

Definitions

- **Analyzing** an algorithm means *predicting the resources that the algorithm requires*.
- Depending on the application; utilization of memory, computer bandwidth, or computer hardware may be of interest.
- But, in general, **computation time** is the main concern.
- Before we can analyze an algorithm, we must have a model of the implementation of technology that will be used, including a model for the resources of that technology and their costs.

Analyzing Algorithms

Definitions

- **Analyzing** an algorithm means *predicting the resources that the algorithm requires*.
- Depending on the application; utilization of memory, computer bandwidth, or computer hardware may be of interest.
- But, in general, **computation time** is the main concern.
- Before we can analyze an algorithm, we must have a model of the implementation of technology that will be used, including a model for the resources of that technology and their costs.
- In this class, we will assume a generic one-processor, **random-access machine (RAM)** model of computation as our implementation technology and understand that our algorithms will be implemented as computer programs.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.
- Exponentiation, in general, is NOT a constant time operation.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.
- Exponentiation, in general, is NOT a constant time operation.
- However, we will assume that multiplication by 2^k can be done in constant time as long as k is not larger than the size of a word.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.
- Exponentiation, in general, is NOT a constant time operation.
- However, we will assume that multiplication by 2^k can be done in constant time as long as k is not larger than the size of a word. Why is that so?

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.
- Exponentiation, in general, is NOT a constant time operation.
- However, we will assume that multiplication by 2^k can be done in constant time as long as k is not larger than the size of a word. Why is that so? Many computers have a shift-left instruction, which in constant time shifts the bits of an integer by k positions to the left and in most computers, shifting the bits of an integer by one position to the left amount to multiplying by 2.

RAM Model

Details

- In the RAM model, instructions are executed one after another, with **no concurrent operations**.
- **Arithmetic** (add, subtract, multiply, divide, remainder, floor, ceiling), **data movement** (load, store, copy), and **control operations** (conditional and unconditional branch, subroutine call and return) can be done in constant time.
- Exponentiation, in general, is NOT a constant time operation.
- However, we will assume that multiplication by 2^k can be done in constant time as long as k is not larger than the size of a word. Why is that so? Many computers have a shift-left instruction, which in constant time shifts the bits of an integer by k positions to the left and in most computers, shifting the bits of an integer by one position to the left amounts to multiplying by 2.
- In the RAM model, we do not attempt to model the memory hierarchy that is common in contemporary computers, i.e., we do not model caches or virtual memory, in order not to complicate things. After all, RAM-model analyses are usually excellent predictors of performance on actual machines.

Analysis of Insertion Sort

Pseudocode

```
INSERTION-SORT (A)
for  $i = 2$  to  $\text{length}[A]$ 
     $\text{key} = A[i];$ 
     $j = i - 1;$ 

    while  $j > 0$  and  $A[j] > \text{key}$ 
         $A[j + 1] = A[j];$ 
         $j = j - 1;$ 

     $A[j + 1] = \text{key};$ 
```

Analysis of Insertion Sort

Pseudocode

```
INSERTION-SORT (A)
for  $i = 2$  to  $\text{length}[A]$ 
     $\text{key} = A[i];$ 
     $j = i - 1;$ 

    while  $j > 0$  and  $A[j] > \text{key}$ 
         $A[j + 1] = A[j];$ 
         $j = j - 1;$ 

     $A[j + 1] = \text{key};$ 
```

Analysis

Let $c_1 \dots c_7$ denote the costs of executing each line of the pseudocode respectively. Then, for an array of size n , the time to execute this algorithm is given by

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^n t_i + c_5 \sum_{i=2}^n (t_i - 1) + c_6 \sum_{i=2}^n (t_i - 1) + c_7(n-1),$$
 where t_i denotes the number of times the while loop test is executed for that value of i .

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \ \forall i \in \{2, 3, \dots, n\}$.

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \forall i \in \{2, 3, \dots, n\}$.
- Thus, the worst-case running-time is

$$T(n) = \left(\frac{c_4 + c_5 + c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7)n - (c_2 + c_3 + c_4 + c_7).$$

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \forall i \in \{2, 3, \dots, n\}$.
- Thus, the worst-case running-time is
$$T(n) = \left(\frac{c_4 + c_5 + c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7)n - (c_2 + c_3 + c_4 + c_7).$$
- **Best Case:** The array is already sorted in ascending order. In this case, $t_i = 1 \forall i$.

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \forall i \in \{2, 3, \dots, n\}$.
- Thus, the worst-case running-time is
$$T(n) = \left(\frac{c_4 + c_5 + c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7)n - (c_2 + c_3 + c_4 + c_7).$$
- **Best Case:** The array is already sorted in ascending order. In this case, $t_i = 1 \forall i$.
- Thus, the best-case running time is $T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7).$

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \forall i \in \{2, 3, \dots, n\}$.
- Thus, the worst-case running-time is

$$T(n) = \left(\frac{c_4 + c_5 + c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7)n - (c_2 + c_3 + c_4 + c_7).$$
- **Best Case:** The array is already sorted in ascending order. In this case, $t_i = 1 \forall i$.
- Thus, the best-case running time is $T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7)$.
- We will say that the insertion sort runs in $\Theta(n^2)$ -time in the worst-case, and it runs in $\Theta(n)$ -time in the best case. We will ignore the lower-order terms and the constants.

Analysis Continued

Worst, Best, Average

- **Worst Case:** The array is sorted in descending order. In this case $t_i = i \forall i \in \{2, 3, \dots, n\}$.
- Thus, the worst-case running-time is

$$T(n) = \left(\frac{c_4 + c_5 + c_6}{2}\right)n^2 + (c_1 + c_2 + c_3 + \frac{c_4 - c_5 - c_6}{2} + c_7)n - (c_2 + c_3 + c_4 + c_7).$$
- **Best Case:** The array is already sorted in ascending order. In this case, $t_i = 1 \forall i$.
- Thus, the best-case running time is $T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7)$.
- We will say that the insertion sort runs in $\Theta(n^2)$ -time in the worst-case, and it runs in $\Theta(n)$ -time in the best case. We will ignore the lower-order terms and the constants.
- What can we say about the time complexity if we first shuffle the array such that the probability of an element appearing in any location in the array is exactly $\frac{1}{n}$?

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*
- **Example 1:** If our program is to test a given integer W is prime or not,

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*
- **Example 1:** If our program is to test a given integer W is prime or not, the input size is $\log W$.

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*
- **Example 1:** If our program is to test a given integer W is prime or not, the input size is $\log W$.
- **Example 2:** If our program is sorting an array of n integers,

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*
- **Example 1:** If our program is to test a given integer W is prime or not, the input size is $\log W$.
- **Example 2:** If our program is sorting an array of n integers, the input size is n .

Analysis of an Algorithm

General Information

- The time taken by an algorithm depends on the input: sorting a thousand numbers takes longer than sorting three numbers.
- The running time of an algorithm may differ on two different inputs of the same size: Insertion sort takes less time if the input is already sorted.
- In general, **the time taken by an algorithm grows with the size of the input** so it is traditional to describe the running time of a program as a **function of the size of its input**.
- **Size of the input:** *The total number of bits needed to represent the input!*
- **Example 1:** If our program is to test a given integer W is prime or not, the input size is $\log W$.
- **Example 2:** If our program is sorting an array of n integers, the input size is n .
- Sometimes, we describe the size of the input with two numbers rather than one. For instance, if the input is a graph, the input size can be described by the number of vertices and the number of edges.

Self-Study Exercises

Exercise 1

Suppose we are comparing implementations of insertion sort and merge sort on the same machine. For inputs of size n , insertion sort runs in $8n^2$ steps, while merge sort runs in $64n \lg n$ steps. For which values of n does insertion sort beat merge sort?

Self-Study Exercises

Exercise 1

Suppose we are comparing implementations of insertion sort and merge sort on the same machine. For inputs of size n , insertion sort runs in $8n^2$ steps, while merge sort runs in $64n \lg n$ steps. For which values of n does insertion sort beat merge sort?

Exercise 2

What is the smallest value of n such that an algorithm whose running time is $100n^2$ runs faster than an algorithm whose running time is 2^n *on the same machine*?

Exercises

Exercise 3

Express the function $\frac{n^3}{1000} - 100n^2 - 100n + 3$ in terms of Θ -notation.

Exercises

Exercise 3

Express the function $\frac{n^3}{1000} - 100n^2 - 100n + 3$ in terms of Θ -notation.

Exercise 4

Selection Sort: Consider sorting n numbers of stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n - 1$ elements of A .

- Write pseudocode for this algorithm.

Exercises

Exercise 3

Express the function $\frac{n^3}{1000} - 100n^2 - 100n + 3$ in terms of Θ -notation.

Exercise 4

Selection Sort: Consider sorting n numbers of stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n - 1$ elements of A

- Write pseudocode for this algorithm.
- What loop invariant does this algorithm maintain?

Exercises

Exercise 3

Express the function $\frac{n^3}{1000} - 100n^2 - 100n + 3$ in terms of Θ -notation.

Exercise 4

Selection Sort: Consider sorting n numbers of stored in array A by first finding the smallest element of A and exchanging it with the element in $A[1]$. Then find the second smallest element of A , and exchange it with $A[2]$. Continue in this manner for the first $n - 1$ elements of A .

- Write pseudocode for this algorithm.
- What loop invariant does this algorithm maintain?
- Why does it need to run for the first $n - 1$ elements, rather than for all n elements?
- Give the worst-case and best-case running times of selection sort in Θ notation.

Exercises Cont.

Exercise 5

In the linear search algorithm, how many elements of the input sequence need to be checked on the average, assuming that the element being searched for is equally likely to be any element in the array? How about the worst case? What are the average case and the worst-case running times of linear search in Θ -notation?

Exercises Cont.

Exercise 5

In the linear search algorithm, how many elements of the input sequence need to be checked on the average, assuming that the element being searched for is equally likely to be any element in the array? How about the worst case? What are the average case and the worst-case running times of linear search in Θ -notation?

Exercise 6

How can we modify almost any algorithm to have a good best-case running time?